Juan Manuel Arias Gutiérrez – 000511986

Juan Andrés Ciro Boada – 000511559

Consultas

1. Plan de ejecución con árbol de acciones:

```
Sort  (cost=133932.99..133935.54 rows=1021 width=63) (actual time=4155.025..4157.231
rows=41793 loops=1)
  Sort Key: streak_summary.weeks_improving DESC, streak_summary.total_points_in_streak
DESC
  Sort Method: quicksort  Memory: 3679kB
  ->  Subquery Scan on streak_summary  (cost=1739.48..133881.96 rows=1021 width=63)
(actual time=32.091..4121.051 rows=41793 loops=1)
        ->  GroupAggregate  (cost=1739.48..133871.75 rows=1021 width=75) (actual
time=32.090..4117.022 rows=41793 loops=1)
              Group Key: artist_streaks.artist_id, artist_streaks.artist_name,
artist_streaks.streak_group
              Filter: (count(*) >= 2)
              Rows Removed by Filter: 50950
              ->  Incremental Sort  (cost=1739.48..133763.14 rows=3133 width=63) (actual
time=32.081..4045.224 rows=166901 loops=1)
                    Sort Key: artist_streaks.artist_id, artist_streaks.artist_name,
artist_streaks.streak_group
                    Presorted Key: artist_streaks.artist_id
                    Full-sort Groups: 1039  Sort Method: quicksort  Average Memory: 27kB  Peak
Memory: 27kB
                    Pre-sorted Groups: 1165  Sort Method: quicksort  Average Memory: 56kB  Peak
Memory: 63kB
                    ->  Subquery Scan on artist_streaks  (cost=1076.26..133657.80 rows=3133 width=63)
(actual time=29.353..3992.097 rows=166901 loops=1)
                          Filter: (artist_streaks.improved = 1)
                          Rows Removed by Filter: 232966
                          ->  WindowAgg  (cost=1076.26..125824.19 rows=626689 width=71) (actual
time=29.349..3955.903 rows=399867 loops=1)
                                ->  Subquery Scan on artist_rank_evolution  (cost=1073.12..111723.68
rows=626689 width=59) (actual time=29.339..3718.327 rows=399867 loops=1)
                                      ->  WindowAgg  (cost=1073.12..111723.68 rows=626689 width=59) (actual
time=29.339..3680.750 rows=399867 loops=1)
                                            ->  Subquery Scan on artist_weekly_rank  (cost=1070.29..100756.63
rows=626689 width=55) (actual time=29.328..3495.535 rows=399867 loops=1)
                                                  ->  GroupAggregate  (cost=1070.29..100756.63 rows=626689
width=55) (actual time=29.327..3458.741 rows=399867 loops=1)
                                                        Group Key: a.artist_id, c.chart_date
                                                        ->  Incremental Sort  (cost=1070.29..86656.12 rows=626689
width=28) (actual time=29.316..3201.606 rows=658255 loops=1)
                                                              Sort Key: a.artist_id, c.chart_date
                                                              Presorted Key: a.artist_id
```

```
                                        Full-sort Groups: 1309  Sort Method: quicksort  Average
Memory: 28kB  Peak Memory: 28kB
                                        Pre-sorted Groups: 1670  Sort Method: quicksort  Average
Memory: 1402kB  Peak Memory: 1517kB
                                        ->  Nested Loop  (cost=1035.02..53236.21 rows=626689
width=28) (actual time=21.665..3045.360 rows=658255 loops=1)
                                            ->  Merge Join  (cost=1034.60..1320.02 rows=12391
width=24) (actual time=18.887..29.311 rows=12391 loops=1)
                                                Merge Cond: (a.artist_id = sa.artist_id)
                                                ->  Index Scan using artist_pkey on artist a
(cost=0.28..94.34 rows=2204 width=15) (actual time=0.900..3.737 rows=2204 loops=1)
                                                ->  Sort  (cost=1034.31..1065.29 rows=12391
width=13) (actual time=17.980..21.064 rows=12391 loops=1)
                                                    Sort Key: sa.artist_id
                                                    Sort Method: quicksort  Memory: 772kB
                                                    ->  Seq Scan on song_artist sa  (cost=0.00..191.91
rows=12391 width=13) (actual time=0.897..15.718 rows=12391 loops=1)
                                            ->  Index Scan using idx_chart_song on chart c
(cost=0.42..3.22 rows=97 width=12) (actual time=0.145..0.237 rows=53 loops=12391)
                                                Index Cond: (song_id = sa.song_id)
Planning Time: 60.143 ms
Execution Time: 4160.037 ms
```

Execution time: 4s 829ms Planning time: 70,4ms Triggers: N/A

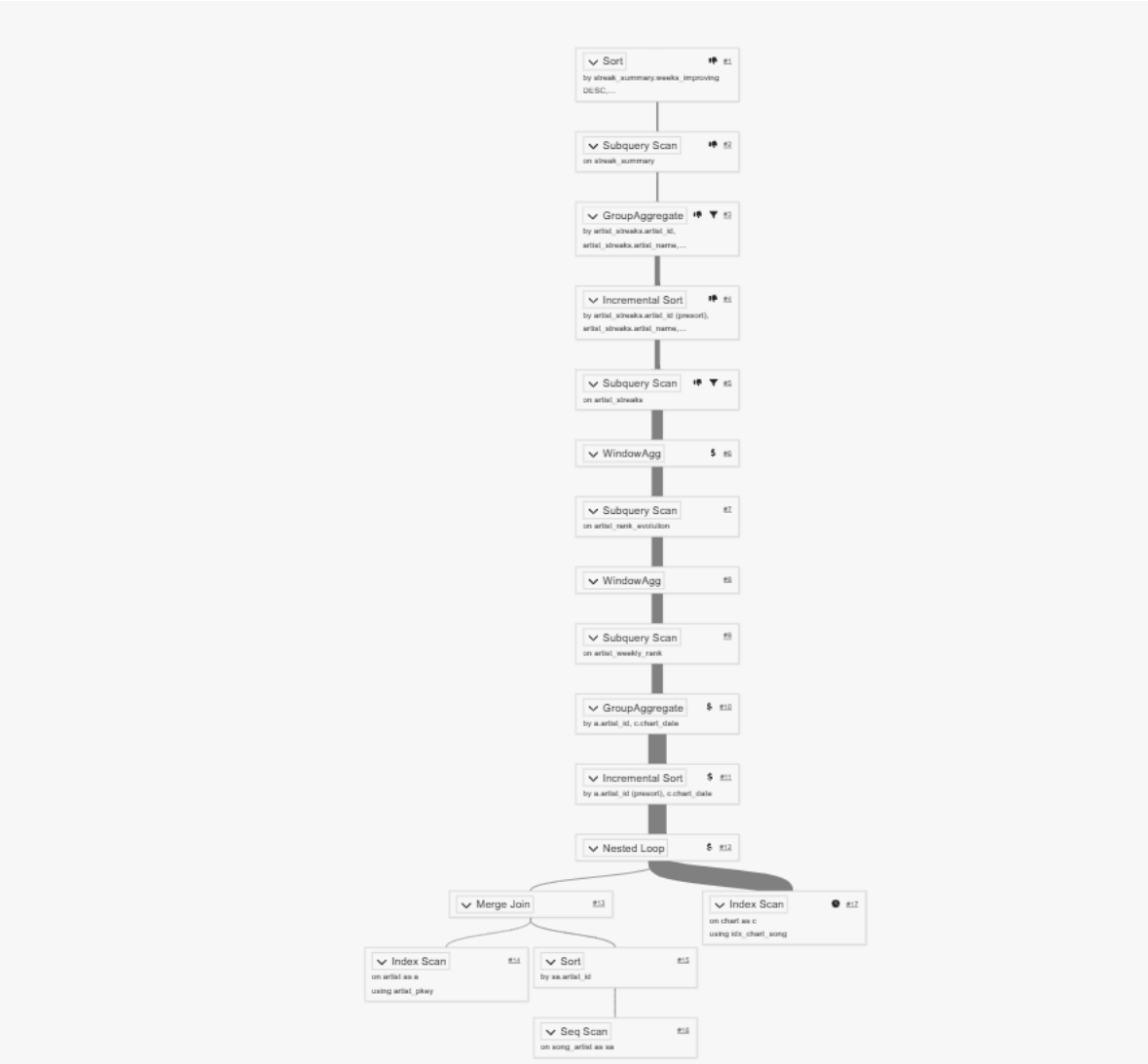
	time	rows	estim	cost	loops	filter	
#1	44	41.793	41 × ▾	53,6			**Sort** by streak_summary.weeks_improving DESC, streak_summary.total_points_in_streak DESC
#2	4	41.793	41 × ▾	10,2			└ **Subquery Scan** on streak_summary
#3	72	41.793	41 × ▾	109		54%	└ **GroupAggregate** by artist_streaks.artist_id, artist_streaks.artist_name, artist_streaks.streak_group
#4	52	166.901	53 × ▾	105			└ **Incremental Sort** by artist_streaks.artist_id (presort), artist_streaks.artist_name, artist_streaks.streak_group
#5	35	166.901	53 × ▾	7830		58%	└ **Subquery Scan** on artist_streaks
#6	235	399.867	1,6 × ▲	14.100			└ **WindowAgg**
#7	37	399.867	1,6 × ▲	0			└ **Subquery Scan** on artist_rank_evolution
#8	178	399.867	1,6 × ▲	11.000			└ **WindowAgg**
#9	41	399.867	1,6 × ▲	0			└ **Subquery Scan** on artist_weekly_rank
#10	260	399.867	1,6 × ▲	14.100			└ **GroupAggregate** by a.artist_id, c.chart_date
#11	155	658.255	1,1 × ▾	33.400			└ **Incremental Sort** by a.artist_id (presort), c.chart_date
#12	77	658.255	1,1 × ▾	51.900			└ **Nested Loop**
#13	4	12.391		160			├ **Merge Join**
#14	4	2204		94,3			│ ├ **Index Scan** on artist as a using artist_pkey
#15	6	12.391		873			│ └ **Sort** by sa.artist_id
#16	14	12.391		192			│ └ **Seq Scan** on song_artist as sa
#17	3606	~656.723	1,8 × ▲	3,22	12.391		└ **Index Scan** on chart as c using idx_chart_song

Table	Count	Time ⇵	
chart	1	**2s 937ms**	**71%**
#17 Index Scan		2s 937ms	71%
song_artist	1	**15,7ms**	**0%**
#16 Seq Scan		15,7ms	0%
artist	1	**3,74ms**	**0%**
#14 Index Scan		3,74ms	0%

Function	Count	Time ⇵
No function used		

Node Type	Count	Time ⇵
Index Scan	2	**2s 940ms** **71%**
#17 Index Scan		2s 937ms 71%
#14 Index Scan		3,74ms 0%
WindowAgg	2	**423ms** **10%**
#8 WindowAgg		238ms 6%
#6 WindowAgg		185ms 4%
GroupAggregate	2	**329ms** **8%**
#10 GroupAggregate		257ms 6%
#3 GroupAggregate		71,8ms 2%
Incremental Sort	2	**209ms** **5%**
#11 Incremental Sort		155ms 4%
#4 Incremental Sort		53,1ms 1%
Subquery Scan	4	**115ms** **3%**
#7 Subquery Scan		37,6ms 1%
#9 Subquery Scan		36,8ms 1%
#5 Subquery Scan		36,2ms 1%
#2 Subquery Scan		4,03ms 0%
Nested Loop	1	**79,4ms** **2%**
#12 Nested Loop		79,4ms 2%
Sort	2	**41,5ms** **1%**
#1 Sort		36,2ms 1%
#15 Sort		5,35ms 0%
Seq Scan	1	**15,7ms** **0%**
#16 Seq Scan		15,7ms 0%
Merge Join	1	**4,51ms** **0%**
#13 Merge Join		4,51ms 0%

Index	Count	Time ⇵	
idx_chart_song	1	**2s 937ms**	**71%**
#17 Index Scan		2s 937ms	71%
artist_pkey	1	**3,74ms**	**0%**
#14 Index Scan		3,74ms	0%



La operación más costosa es el **GroupAggregate**, ya que procesa un alto volumen de datos (aproximadamente 399,867 filas) provenientes de un resultado intermedio de cerca de 658,255 registros, y además requiere un ordenamiento previo por artist_id y chart_date (Incremental Sort), lo que incrementa significativamente el tiempo de ejecución. Este costo está influenciado por el diseño del modelo y los índices, donde el índice sobre *chart(*song_id*)* mejora el acceso a esa tabla.

Resultado de la consulta:

	artist_name	weeks_improving	total_points_in_streak	streak_start	streak_end	best_rank_in_streak
1	Anitta	18	3510	2022-03-07	2022-03-24	1
2	G-Eazy	18	2700	2017-04-08	2017-04-25	30
3	Surf Mesa	17	3247	2020-03-30	2020-04-15	38
4	Jos Blue	17	2160	2017-05-15	2017-06-03	21
5	Cardi B	17	2159	2017-08-08	2017-08-24	39
6	William Singe	17	1156	2017-05-15	2017-06-03	21
7	KHEA	17	400	2018-04-14	2019-03-16	40
8	Nicki Nicole	16	432	2021-02-12	2021-02-27	25
9	Rafa Pabön	16	414	2019-03-01	2019-03-16	40
10	Farruko	15	2910	2021-07-17	2021-07-31	14
11	Alesso	15	1245	2018-09-04	2018-09-18	50
12	Regard	15	1155	2019-09-07	2019-09-21	60
13	Jos Blue	15	960	2018-06-04	2018-06-18	63
14	Rochy RD	15	840	2021-02-13	2021-02-27	25
15	Oliver Tree	14	2576	2021-09-03	2021-09-16	21
16	Tyga	14	1708	2018-05-19	2018-06-01	44
17	Brando	14			2018-07-18	26
18	Loud Luxury	14	1232	2018-07-05	2018-07-18	26

1-500 of 501+

y_charts_db > console 76:60 CRLF UTF-8 4 spaces Non-commercial use

2. Plan de ejecución con árbol de acciones:

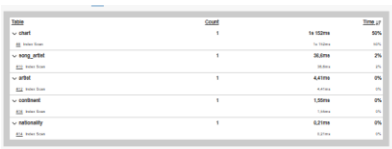
```
Incremental Sort  (cost=128289.11..267429.81 rows=626689 width=326) (actual
time=8.374..1395.183 rows=13921 loops=1)
  Sort Key: cwp.chart_date, (rank() OVER (?))
  Presorted Key: cwp.chart_date
  Full-sort Groups: 392  Sort Method: quicksort  Average Memory: 27kB  Peak Memory: 27kB
  CTE continent_weekly_points
    ->  GroupAggregate  (cost=47.06..127267.00 rows=626689 width=258) (actual
time=1.461..1363.688 rows=13921 loops=1)
          Group Key: c.chart_date, co.continent_id
          ->  Incremental Sort  (cost=47.06..114733.22 rows=626689 width=231) (actual
time=1.428..1265.437 rows=658255 loops=1)
                Sort Key: c.chart_date, co.continent_id
                Presorted Key: c.chart_date
                Full-sort Groups: 2336  Sort Method: quicksort  Average Memory: 27kB  Peak
Memory: 27kB
                Pre-sorted Groups: 2336  Sort Method: quicksort  Average Memory: 36kB  Peak
Memory: 36kB
                ->  Nested Loop  (cost=1.32..81573.62 rows=626689 width=231) (actual
time=0.042..1125.597 rows=658255 loops=1)
                      ->  Nested Loop  (cost=1.16..65928.79 rows=626689 width=13) (actual
time=0.037..916.643 rows=658255 loops=1)
                            ->  Nested Loop  (cost=1.01..50380.36 rows=626689 width=13) (actual
time=0.034..714.816 rows=658255 loops=1)
                                  ->  Nested Loop  (cost=0.72..34043.64 rows=626689 width=13) (actual
time=0.026..499.785 rows=658255 loops=1)
                                        ->  Index Scan using idx_chart_date on chart c  (cost=0.42..20942.44
rows=465979 width=8) (actual time=0.012..222.516 rows=465979 loops=1)
```

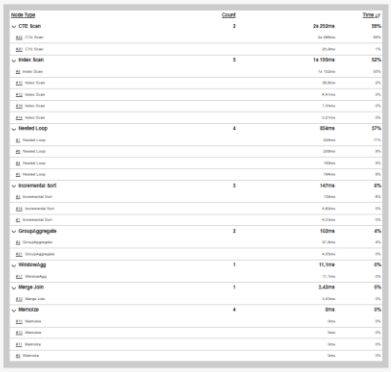
```
                                -> Memoize (cost=0.30..0.31 rows=1 width=13) (actual
time=0.000..0.000 rows=1 loops=465979)
                                      Cache Key: c.song_id
                                      Cache Mode: logical
                                      Hits: 456819  Misses: 9160  Evictions: 0  Overflows: 0  Memory Usage:
1190kB
                                      -> Index Scan using song_artist_pk on song_artist sa
(cost=0.29..0.30 rows=1 width=13) (actual time=0.002..0.002 rows=1 loops=9160)
                                            Index Cond: (song_id = c.song_id)
                           -> Memoize (cost=0.29..0.32 rows=1 width=8) (actual time=0.000..0.000
rows=1 loops=658255)
                                Cache Key: sa.artist_id
                                Cache Mode: logical
                                Hits: 656051  Misses: 2204  Evictions: 0  Overflows: 0  Memory Usage:
233kB
                                -> Index Scan using artist_pkey on artist a (cost=0.28..0.31 rows=1
width=8) (actual time=0.002..0.002 rows=1 loops=2204)
                                      Index Cond: (artist_id = sa.artist_id)
                        -> Memoize (cost=0.15..0.18 rows=1 width=8) (actual time=0.000..0.000
rows=1 loops=658255)
                            Cache Key: a.nationality_id
                            Cache Mode: logical
                            Hits: 658185  Misses: 70  Evictions: 0  Overflows: 0  Memory Usage: 8kB
                            -> Index Scan using nationality_pkey on nationality n (cost=0.14..0.17
rows=1 width=8) (actual time=0.002..0.002 rows=1 loops=70)
                                Index Cond: (nationality_id = a.nationality_id)
                     -> Memoize (cost=0.16..0.86 rows=1 width=222) (actual time=0.000..0.000 rows=1
loops=658255)
                        Cache Key: n.continent_id
                        Cache Mode: logical
                        Hits: 658248  Misses: 7  Evictions: 0  Overflows: 0  Memory Usage: 1kB
                        -> Index Scan using continent_pkey on continent co (cost=0.15..0.85 rows=1
width=222) (actual time=0.001..0.001 rows=1 loops=7)
                            Index Cond: (continent_id = n.continent_id)
 -> WindowAgg (cost=362.29..95934.82 rows=626689 width=326) (actual time=5.331..1391.342
rows=13921 loops=1)
      -> Incremental Sort (cost=362.15..80267.60 rows=626689 width=286) (actual
time=5.323..1380.504 rows=13921 loops=1)
           Sort Key: cwp.chart_date, cwp.continent_points DESC
           Presorted Key: cwp.chart_date
           Full-sort Groups: 392  Sort Method: quicksort  Average Memory: 26kB  Peak Memory:
26kB
           -> Merge Join (cost=0.00..36039.62 rows=626689 width=286) (actual
time=2.073..1375.969 rows=13921 loops=1)
                Merge Cond: (cwp.chart_date = continent_weekly_points.chart_date)
                -> CTE Scan on continent_weekly_points cwp (cost=0.00..12533.78 rows=626689
width=254) (actual time=1.463..2.573 rows=13921 loops=1)
```

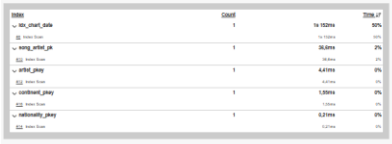
```
                -> GroupAggregate (cost=0.00..15669.73 rows=200 width=36) (actual time=0.607..1370.330 rows=2336 loops=1)
                      Group Key: continent_weekly_points.chart_date
                      -> CTE Scan on continent_weekly_points (cost=0.00..12533.78 rows=626689 width=36) (actual time=0.000..1366.638 rows=13921 loops=1)
Planning Time: 0.617 ms
Execution Time: 1396.042 ms
```

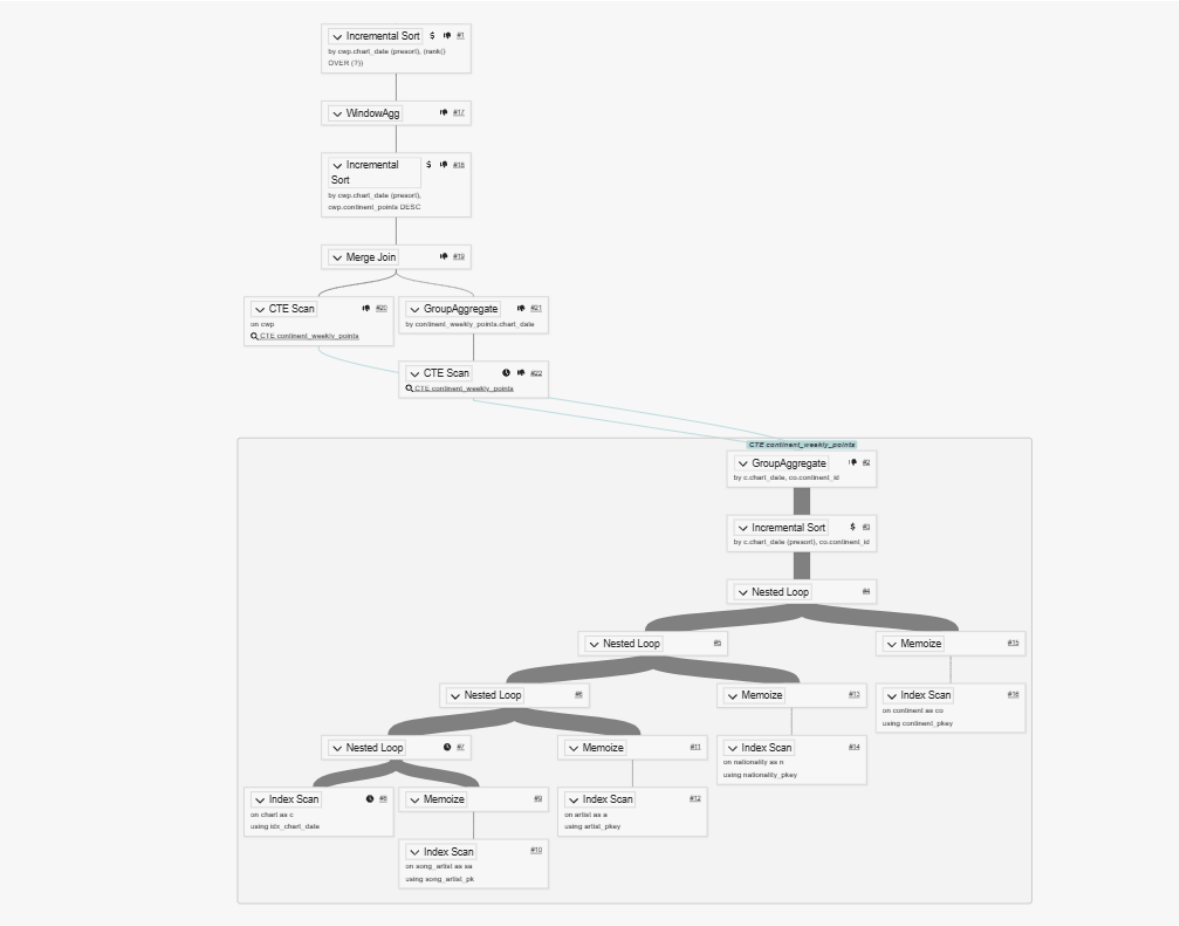
Execution time: 2s 320ms Planning time: 29,9ms Triggers: N/A

	time	rows	estim	cost	loops	
#1	4	13.921	45 ×	171.000		**Incremental Sort** by cwp.chart_date (presort), (rank() OVER (?))
#17	11	13.921	45 ×	15.700		└ **WindowAgg**
#18	5	13.921	45 ×	44.200		└ **Incremental Sort** by cwp.chart_date (presort), cwp.continent_points DESC
#19	3	13.921	45 ×	7840		└ **Merge Join**
#20	26	13.921	45 ×	12.500		├ **CTE Scan** on cwp CTE continent_weekly_points
#21	4	2336	12 ×	3140		└ **GroupAggregate** by continent_weekly_points.chart_date
#22	2266	13.921	45 ×	12.500		└ **CTE Scan** CTE continent_weekly_points
						CTE continent_weekly_points
#2	98	13.921	45 ×	12.500		**GroupAggregate** by c.chart_date, co.continent_id
#3	138	658.255	1,1 ×	33.200		└ **Incremental Sort** by c.chart_date (presort), co.continent_id
#4	199	658.255	1,1 ×	15.600		└ **Nested Loop**
#5	194	658.255	1,1 ×	15.500		├ **Nested Loop**
#6	208	658.255	1,1 ×	16.300		│ ├ **Nested Loop**
#7	254	658.255	1,1 ×	13.100		│ │ ├ **Nested Loop**
#8	1152	465.979		20.900		│ │ │ ├ **Index Scan** on chart as c using idx_chart_date
#9	0	~465.979		0,01	465.979	│ │ │ └ **Memoize**
#10	37	~9160		0,3	9160	│ │ │ └ **Index Scan** on song_artist as sa using song_artist_pk
#11	0	~658.255		0,01	658.255	│ │ └ **Memoize**
#12	4	~2204		0,31	2204	│ │ └ **Index Scan** on artist as a using artist_pkey
#13	0	~658.255		0,01	658.255	│ └ **Memoize**
#14	0	~70		0,17	70	│ └ **Index Scan** on nationality as n using nationality_pkey
#15	0	~658.255		0,01	658.255	└ **Memoize**
#16	2	~7		0,85	7	└ **Index Scan** on continent as co using continent_pkey









La operación más costosa es el GroupAggregate, ya que agrupa datos por chart_date y continent_id procesando un volumen intermedio alto (aprox. 658,255 filas) producto de múltiples joins, lo que requiere un ordenamiento previo (Incremental Sort) que incrementa el tiempo de ejecución. Además, el CTE es escaneado dos veces, duplicando su costo de procesamiento. El diseño del modelo y los índices influyen positivamente en el rendimiento, ya que el índice sobre chart(chart_date) permite un acceso ordenado y reduce el costo del sort, mientras que mecanismos como la caché en los joins disminuyen el impacto del alto número de accesos.

Resultado de la consulta:

Output | Result 2

CSV

	continent_name	chart_date	continent_points	global_points	participation_pct	weekly_rank
1	Anglo-America	2017-01-01	6922	11927	58.04	1
2	Europe	2017-01-01	3372	11927	28.27	2
3	Latin-America	2017-01-01	1205	11927	10.1	3
4	Oceania	2017-01-01	348	11927	2.92	4
5	Asia	2017-01-01	57	11927	0.48	5
6	Africa	2017-01-01	23	11927	0.19	6
7	Anglo-America	2017-01-02	6614	11815	55.98	1
8	Europe	2017-01-02	3661	11815	30.99	2
9	Latin-America	2017-01-02	1112	11815	9.41	3
10	Oceania	2017-01-02	348	11815	2.95	4
11	Asia	2017-01-02	57	11815	0.48	5
12	Africa	2017-01-02	23	11815	0.19	6
13	Anglo-America	2017-01-03	6723	11991	56.07	1
14	Europe	2017-01-03	3645	11991	30.4	2
15	Latin-America	2017-01-03	1066	11991	8.89	3
16	Oceania	2017-01-03		11991	3.98	4
17	Asia	2017-01-03	57	11991	0.48	5

1-500 of 501+